CHAPTER 3

DESIGN AND ARCHITECTURE

3.1 Overall Design

3.2 Arithmetic Logic Unit (8-bit ALU)

3.3 Registers

3.4 Multiplexers and Decoders

3.5 Control Unit

3.6 RAM and ROM

3.7 Data Bus and Address Bus

Chapter: 3

DESIGN AND ARCHITECTURE

This chapter describes the design and architecture of the proposed 8-bit CPU. It is based on the uploaded project report and follows the structure of the senior report while describing the user's own project.

3.1 Overall Design

The proposed 8-bit CPU follows a modular architecture consisting of an Arithmetic Logic Unit (ALU), general-purpose registers, accumulator, program counter, instruction register, control unit, RAM, ROM, multiplexers, and common data and address buses. During execution, instructions are fetched from memory, decoded by the control unit, and executed by the ALU. Registers temporarily store operands and intermediate results, while memory stores instructions and data. A common clock synchronizes all modules to ensure reliable operation. This educational processor demonstrates the complete fetch–decode–execute cycle and provides a practical understanding of computer architecture.

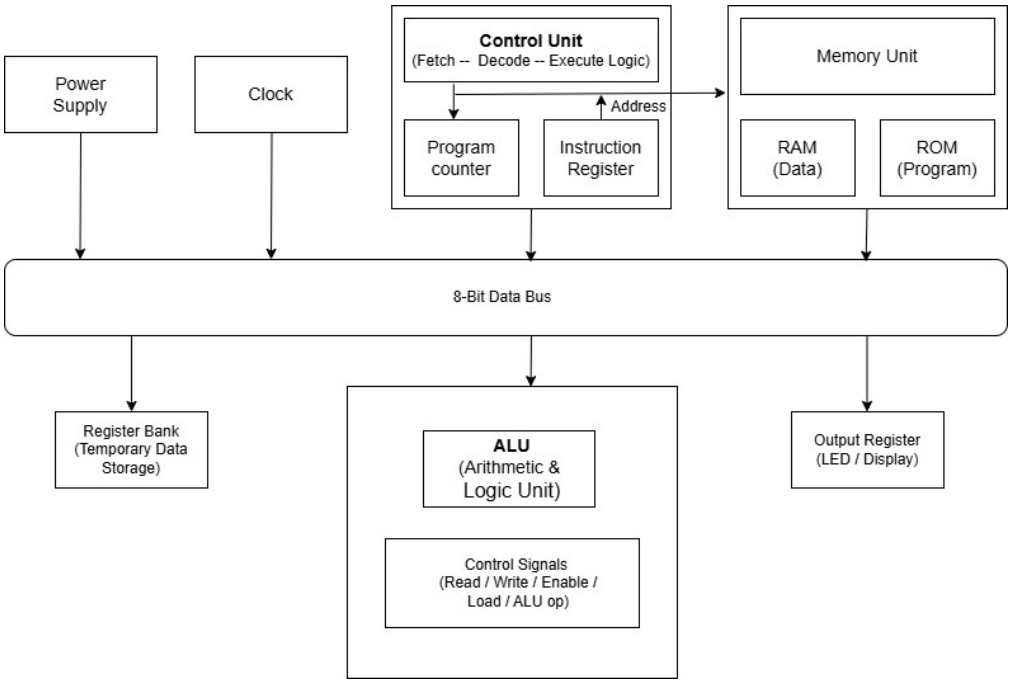


Figure 3.1 Overall Architecture of 8-bit CPU

The proposed 8-bit CPU follows a modular architecture consisting of an Arithmetic Logic Unit (ALU), general-purpose registers, accumulator, program counter, instruction register, control unit, RAM, ROM, multiplexers, and common data and address buses. During execution, instructions are fetched from memory, decoded by the control unit, and executed by the ALU. Registers temporarily store operands and intermediate results, while memory stores instructions and data. A common clock synchronizes all modules to ensure reliable operation. This educational processor demonstrates the complete fetch–decode–execute cycle and provides a practical understanding of computer architecture

The proposed 8-bit CPU follows a modular architecture consisting of an Arithmetic Logic Unit (ALU), general-purpose registers, accumulator, program counter, instruction register, control

unit, RAM, ROM, multiplexers, and common data and address buses. During execution, instructions are fetched from memory, decoded by the control unit, and executed by the ALU. Registers temporarily store operands and intermediate results, while memory stores instructions and data. A common clock synchronizes all modules to ensure reliable operation. This educational processor demonstrates the complete fetch–decode–execute cycle and provides a practical understanding of computer architecture.

The proposed 8-bit CPU follows a modular architecture consisting of an Arithmetic Logic Unit (ALU), general-purpose registers, accumulator, program counter, instruction register, control unit, RAM, ROM, multiplexers, and common data and address buses. During execution, instructions are fetched from memory, decoded by the control unit, and executed by the ALU. Registers temporarily store operands and intermediate results, while memory stores instructions and data. A common clock synchronizes all modules to ensure reliable operation. This educational processor demonstrates the complete fetch–decode–execute cycle and provides a practical understanding of computer architecture.

The proposed 8-bit CPU follows a modular architecture consisting of an Arithmetic Logic Unit (ALU), general-purpose registers, accumulator, program counter, instruction register, control unit, RAM, ROM, multiplexers, and common data and address buses. During execution, instructions are fetched from memory, decoded by the control unit, and executed by the ALU. Registers temporarily store operands and intermediate results, while memory stores instructions and data. A common clock synchronizes all modules to ensure reliable operation. This educational processor demonstrates the complete fetch–decode–execute cycle and provides a practical understanding of computer architecture.

The proposed 8-bit CPU follows a modular architecture consisting of an Arithmetic Logic Unit (ALU), general-purpose registers, accumulator, program counter, instruction register, control unit, RAM, ROM, multiplexers, and common data and address buses. During execution, instructions are fetched from memory, decoded by the control unit, and executed by the ALU. Registers temporarily store operands and intermediate results, while memory stores instructions and data. A common clock synchronizes all modules to ensure reliable operation. This educational processor demonstrates the complete fetch–decode–execute cycle and provides a practical understanding of computer architecture.

The proposed 8-bit CPU follows a modular architecture consisting of an Arithmetic Logic Unit (ALU), general-purpose registers, accumulator, program counter, instruction register, control unit, RAM, ROM, multiplexers, and common data and address buses. During execution, instructions are fetched from memory, decoded by the control unit, and executed by the ALU. Registers temporarily store operands and intermediate results, while memory stores instructions and data. A common clock synchronizes all modules to ensure reliable operation. This educational processor demonstrates the complete fetch–decode–execute cycle and provides a practical understanding of computer architecture.

The proposed 8-bit CPU follows a modular architecture consisting of an Arithmetic Logic Unit (ALU), general-purpose registers, accumulator, program counter, instruction register, control unit, RAM, ROM, multiplexers, and common data and address buses. During execution, instructions are fetched from memory, decoded by the control unit, and executed by the ALU. Registers temporarily store operands and intermediate results, while memory stores instructions and data. A common clock synchronizes all modules to ensure reliable operation.

This educational processor demonstrates the complete fetch–decode–execute cycle and provides a practical understanding of computer architecture.

3.2 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) is a crucial component of the Anviksha CPU, responsible for executing arithmetic and logical operations. The ALU's construction begins with basic logic gates.

Table 3.2: Truth Table of 1-bit Full adder

INPUT A	INPUT B	INPUT Cin	OUTPUT Sum	OUTPUT Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

The truth table for a 1-bit full adder, which shows the relationships between the inputs (A, B, and Cin) and the outputs (Sum and Carry). A full adder is a fundamental digital circuit used in arithmetic operations, specifically for adding three bits: two operand bits (A and B) and a carry-in bit (Cin). The outputs are a sum bit (Sum) and a carry-out bit (Carry). The table details all possible combinations of these inputs and their corresponding outputs. For example, when A and B are both 0 and Cin is 1, the sum is 1, and the carry is 0. When A is 1, B is 1, and Cin is 1, the sum is 1, and the carry is also 1. This truth table highlights the essential function of the full adder in binary addition, where it computes the least significant bit (Sum) and determines whether a carry needs to be added to the next significant bit (Carry). Understanding this table is crucial for grasping how multiple full adders can be combined to create multi-bit adders, such as an 8-bit adder, integral to CPU operations for performing complex arithmetic operations across full 8-bit registers.

Designing an 8-Bit Adder

An 8-bit adder is designed to add two 8-bit binary numbers (A and B) along with a carry-in (Cin) and produce an 8-bit sum (S) and a carry-out (Cout). It is constructed by cascading eight 1-bit full adders in a ripple-carry configuration, where the carry generated by one stage is passed to the next stage.

Adders are fundamental digital circuits used extensively in computer processors, particularly within the Arithmetic Logic Unit (ALU), to perform arithmetic operations on binary numbers. They are essential components that enable various arithmetic and logical computations in computing systems.

The primary function of an 8-bit adder is to compute the sum of two 8-bit binary numbers. It accepts two 8-bit operands and an optional carry-in bit, producing an 8-bit sum and a carry-out bit. The adder can handle both unsigned and signed binary numbers using representations such as two's complement, making it suitable for a wide range of arithmetic operations.

Operation

The operation of an 8-bit binary adder involves adding two 8-bit binary numbers, A and B, along with a carry-in (Cin), to generate an 8-bit sum (S) and a carry-out (Cout).

Each bit position is processed by a 1-bit full adder. The least significant bit (LSB) receives the external carry-in, while each subsequent full adder receives the carry generated by the previous stage. The sum bit for each stage is obtained using XOR operations on the corresponding bits of A, B, and the carry-in. The carry-out for each stage is generated using AND and OR logic gates and is propagated to the next higher bit.

This ripple-carry process continues through all eight stages, ensuring accurate addition of two 8-bit numbers. The final carry-out from the most significant bit (MSB) indicates an overflow in unsigned arithmetic and enables reliable multi-bit arithmetic operations within the 8-bit CPU ALU.

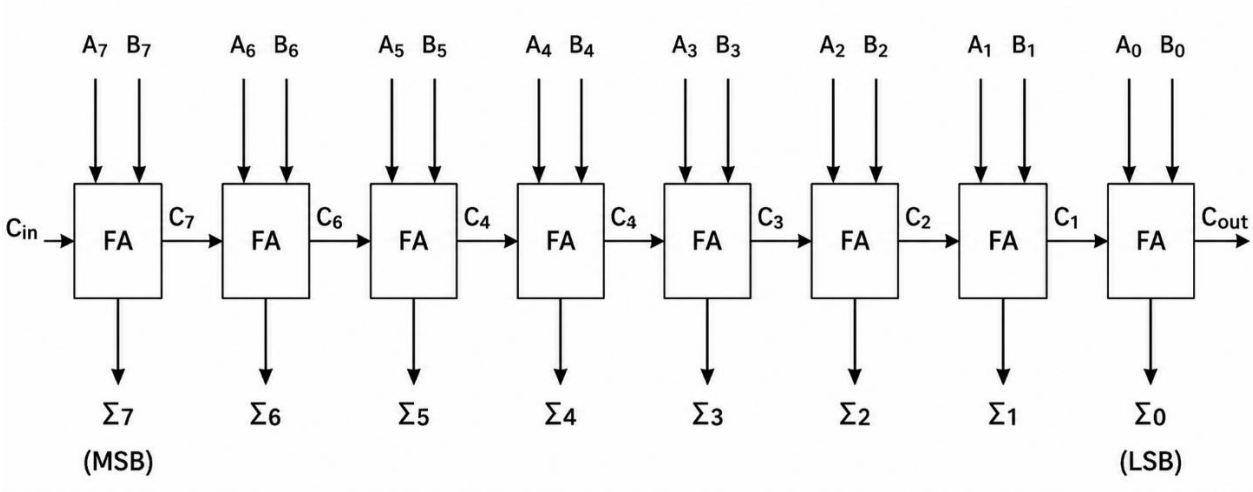


Figure 3.2: 8-bit full adder/ripple carry adder figures as applicable.

3.3 Registers

Registers are temporary storage elements used during instruction execution. The proposed 8-bit CPU includes general-purpose registers, an accumulator, instruction register, and program counter. These registers reduce memory access and improve execution efficiency. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

Registers are temporary storage elements used during instruction execution. The proposed 8-bit CPU includes general-purpose registers, an accumulator, instruction register, and program counter. These registers reduce memory access and improve execution efficiency. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

Registers are temporary storage elements used during instruction execution. The proposed 8-bit CPU includes general-purpose registers, an accumulator, instruction register, and program counter. These registers reduce memory access and improve execution efficiency. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

Registers are temporary storage elements used during instruction execution. The proposed 8-bit CPU includes general-purpose registers, an accumulator, instruction register, and program counter. These registers reduce memory access and improve execution efficiency. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

3.3.1 General Purpose Registers

General-purpose registers are used to store operands, temporary values, and intermediate data required during program execution. These registers provide fast storage locations that allow the CPU to access data quickly without repeatedly communicating with the main memory, thereby improving the overall performance of the processor. In this Logisim-based CPU design, each general-purpose register is capable of storing an 8-bit binary value.

The registers exchange data with the Arithmetic Logic Unit (ALU) through the internal data bus. During an operation, the required operands are transferred from the registers to the ALU, where arithmetic or logical computations are performed. The resulting output is then stored back in a register for further processing or use by other CPU components.

The modular organization of the general-purpose registers simplifies the processor design, making it easier to test, debug, and maintain. The Logisim implementation demonstrates the storage and transfer of data between the registers and the ALU, illustrating their important role in efficient CPU operation.

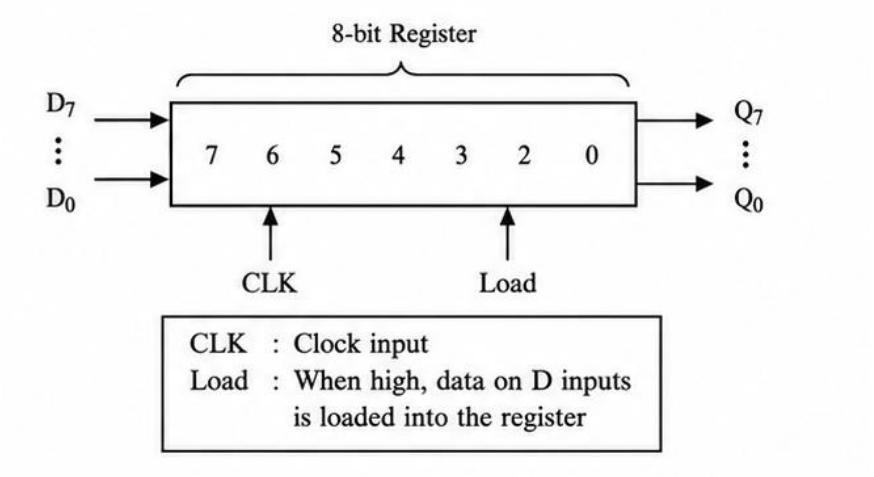


Figure 3.3: General-purpose registers

This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

3.3.2 Accumulator Register

The accumulator is a special-purpose register that stores one of the operands required for arithmetic and logical operations. It also receives and stores the result produced by the Arithmetic Logic Unit (ALU) after each operation. Since many CPU operations involve the accumulator, it serves as a temporary storage location that enables faster data processing and reduces the need for frequent memory access.

In this Logisim-based processor design, the accumulator is updated under the control of the control unit. Appropriate control signals determine when data is loaded into the accumulator and when its contents are transferred to the ALU for processing. After the ALU completes the specified operation, the output is written back to the accumulator for further use or transfer to another register.

The accumulator plays a vital role in the execution of instructions by supporting efficient data transfer and computation. Its modular design simplifies the processor architecture, making the CPU easier to design, test, debug, and maintain. The Logisim implementation demonstrates how the accumulator stores data and interacts with the ALU during processor operations.

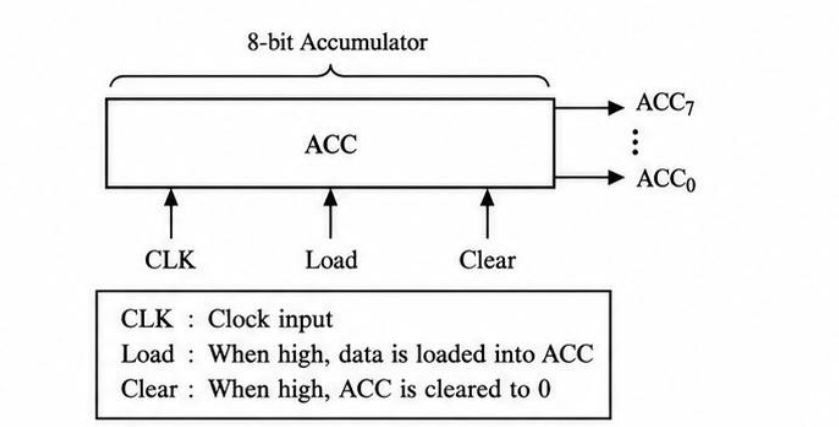


Figure 3.4 Accumulator Register

3.3.3 Instruction Register

The instruction register (IR) is used to store the instruction fetched from memory until it is completely decoded and executed. It temporarily holds the instruction so that the control unit can read and interpret the opcode and other instruction fields. This ensures that the instruction remains available throughout the execution cycle without requiring repeated access to memory.

In this Logisim-based processor design, the instruction register receives the fetched instruction from memory and passes the opcode to the control unit for decoding. Based on the decoded instruction, the control unit generates the necessary control signals to coordinate data transfer and execute the required operation. The instruction register remains unchanged until the next instruction is fetched.

The instruction register plays an important role in the instruction execution process by ensuring accurate decoding and control of CPU operations. Its modular design simplifies the processor architecture, making the system easier to design, test, debug, and maintain. The Logisim implementation demonstrates how instructions are stored and supplied to the control unit during processor operation.

3.3.4 Program Counter

The program counter stores the address of the next instruction. After every fetch cycle it increments unless modified by a jump instruction. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

The program counter stores the address of the next instruction. After every fetch cycle it increments unless modified by a jump instruction. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

The program counter stores the address of the next instruction. After every fetch cycle it increments unless modified by a jump instruction. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

The program counter stores the address of the next instruction. After every fetch cycle it increments unless modified by a jump instruction. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

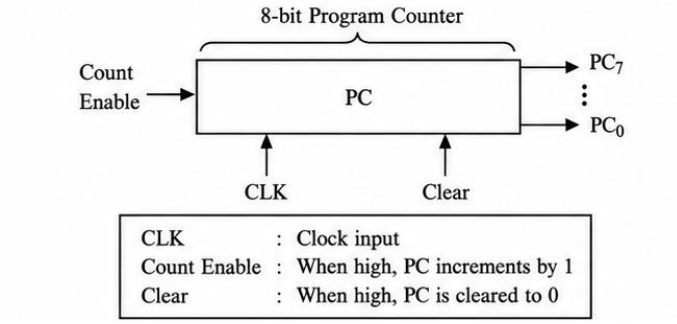


Figure 3.5: Program Counter

3.4 Multiplexer

Multiplexers select one of several data inputs and route it to the required destination under control signals. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

Multiplexers select one of several data inputs and route it to the required destination under control signals. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

Multiplexers select one of several data inputs and route it to the required destination under control signals. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

Multiplexers select one of several data inputs and route it to the required destination under control signals. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

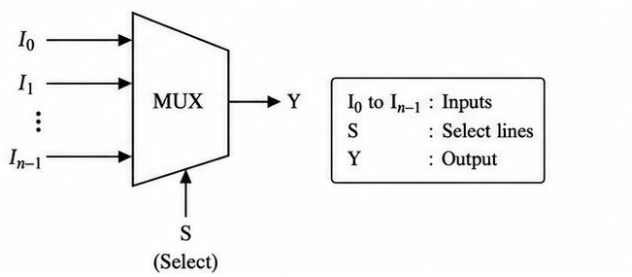


Figure 3.5: Multiplexer

3.4.1 Decoder

The decoder converts opcode bits into individual control signals required by the CPU modules. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

The decoder converts opcode bits into individual control signals required by the CPU modules. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

The decoder converts opcode bits into individual control signals required by the CPU modules. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

The decoder converts opcode bits into individual control signals required by the CPU modules. This modular organization simplifies CPU operation, testing, and debugging. Include the relevant figure and explanation for this module based on your Logisim design.

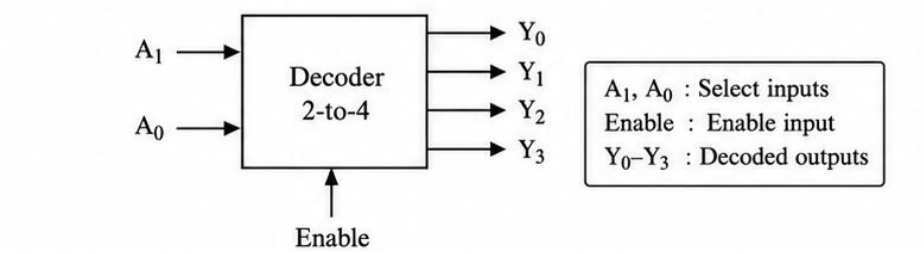


Figure 3.7: Decoder

3.5 Control Unit

The control unit coordinates the operation of all modules in the proposed 8-bit CPU. It interprets the instruction stored in the Instruction Register and generates the required control signals for the ALU, registers, memory, and buses. The control unit ensures that every instruction progresses through the fetch, decode, and execute stages in the correct sequence. Insert the corresponding Logisim architecture diagram and explain the signal flow for this module.

The control unit coordinates the operation of all modules in the proposed 8-bit CPU. It interprets the instruction stored in the Instruction Register and generates the required control signals for the ALU, registers, memory, and buses. The control unit ensures that every instruction progresses through the fetch, decode, and execute stages in the correct sequence.

Insert the corresponding Logisim architecture diagram and explain the signal flow for this module.

The control unit coordinates the operation of all modules in the proposed 8-bit CPU. It interprets the instruction stored in the Instruction Register and generates the required control signals for the ALU, registers, memory, and buses. The control unit ensures that every instruction progresses through the fetch, decode, and execute stages in the correct sequence. Insert the corresponding Logisim architecture diagram and explain the signal flow for this module.

The control unit coordinates the operation of all modules in the proposed 8-bit CPU. It interprets the instruction stored in the Instruction Register and generates the required control signals for the ALU, registers, memory, and buses. The control unit ensures that every instruction progresses through the fetch, decode, and execute stages in the correct sequence. Insert the corresponding Logisim architecture diagram and explain the signal flow for this module.

The control unit coordinates the operation of all modules in the proposed 8-bit CPU. It interprets the instruction stored in the Instruction Register and generates the required control signals for the ALU, registers, memory, and buses. The control unit ensures that every instruction progresses through the fetch, decode, and execute stages in the correct sequence. Insert the corresponding Logisim architecture diagram and explain the signal flow for this module.

3.6 RAM and ROM

RAM stores program data and intermediate results during execution, while ROM stores predefined instructions or initialization code. Both memories communicate with the CPU through the address and data buses. Insert the corresponding Logisim architecture diagram and explain the signal flow for this module.

RAM stores program data and intermediate results during execution, while ROM stores predefined instructions or initialization code. Both memories communicate with the CPU through the address and data buses. Insert the corresponding Logisim architecture diagram and explain the signal flow for this module.

RAM stores program data and intermediate results during execution, while ROM stores predefined instructions or initialization code. Both memories communicate with the CPU through the address and data buses. Insert the corresponding Logisim architecture diagram and explain the signal flow for this module.

RAM stores program data and intermediate results during execution, while ROM stores predefined instructions or initialization code. Both memories communicate with the CPU through the address and data buses. Insert the corresponding Logisim architecture diagram and explain the signal flow for this module.

RAM stores program data and intermediate results during execution, while ROM stores predefined instructions or initialization code. Both memories communicate with the CPU through the address and data buses. Insert the corresponding Logisim architecture diagram and explain the signal flow for this module

Figure 3.8: SRAM 16 Byte Memory (RAM)

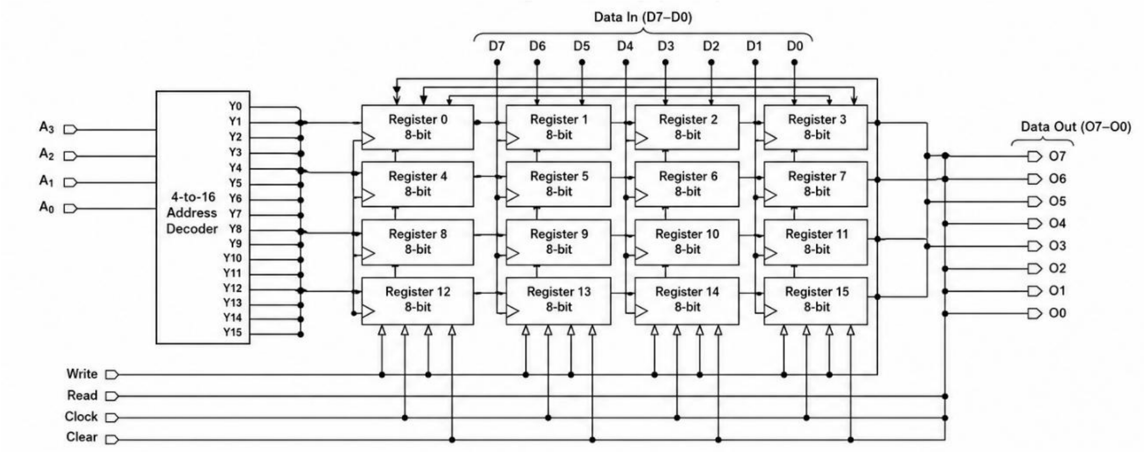
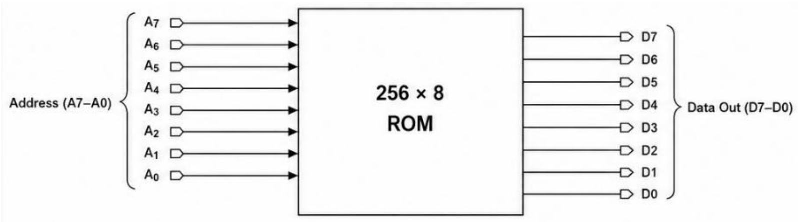


Figure 3.9: ROM 256*8



3.7 Data Bus and Address Bus

The data bus transfers 8-bit data between registers, ALU, and memory. The address bus carries memory addresses from the Program Counter or address generation logic to memory locations. Insert the corresponding Logisim architecture diagram and explain the signal flow for this module.

The data bus transfers 8-bit data between registers, ALU, and memory. The address bus carries memory addresses from the Program Counter or address generation logic to memory locations. Insert the corresponding Logisim architecture diagram and explain the signal flow for this module.

The data bus transfers 8-bit data between registers, ALU, and memory. The address bus carries memory addresses from the Program Counter or address generation logic to memory locations. Insert the corresponding Logisim architecture diagram and explain the signal flow for this module.

The data bus transfers 8-bit data between registers, ALU, and memory. The address bus carries memory addresses from the Program Counter or address generation logic to memory locations. Insert the corresponding Logisim architecture diagram and explain the signal flow for this module.

The data bus transfers 8-bit data between registers, ALU, and memory. The address bus carries memory addresses from the Program Counter or address generation logic to memory locations.

Insert the corresponding Logisim architecture diagram and explain the signal flow for this module.

Chapter Summary

The proposed 8-bit CPU integrates the ALU, registers, program counter, instruction register, control unit, memory, and buses into a modular architecture. The implementation demonstrates the complete instruction cycle and provides an educational platform for understanding computer organization and processor design.